

Лабораторная работа №14

Операционные системы

Мустафина Аделя Юрисовна

Содержание

1	Цель работы	5
2	Задание	6
3	Выполнение лабораторной работы	8
4	Контрольные вопросы	12
5	Выводы	14
6	Список литературы	15

Список иллюстраций

3.1	1		8
3.2	2		9
3.3	3		9
3.4	4		9
3.5	5		9
3.6	6		10
3.7	7		10
3.8	8		10
3.9	9		11

Список таблиц

1 Цель работы

Изучить основы программирования в оболочке ОС UNIX. Научиться писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

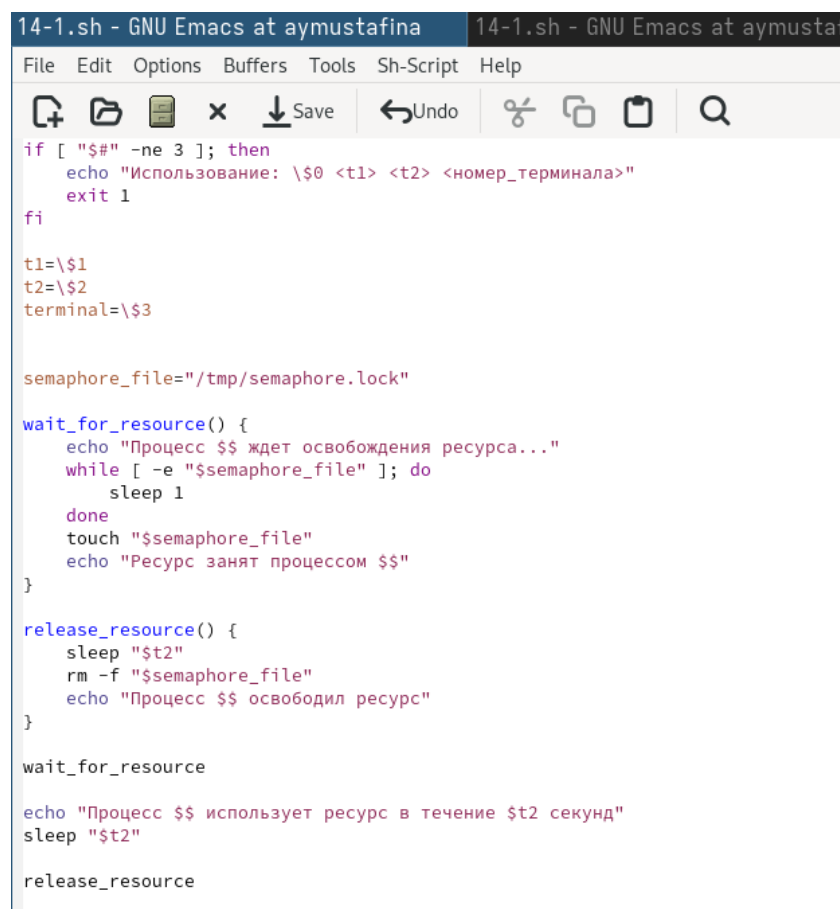
2 Задание

1. Написать командный файл, реализующий упрощённый механизм семафоров. Командный файл должен в течение некоторого времени t_1 дожидаться освобождения ресурса, выдавая об этом сообщение, а дождавшись его освобождения, использовать его в течение некоторого времени $t_2 < t_1$, также выдавая информацию о том, что ресурс используется соответствующим командным файлом (процессом). Запустить командный файл в одном виртуальном терминале в фоновом режиме, перенаправив его вывод в другой (`> /dev/tty#`, где `#` — номер терминала куда перенаправляется вывод), в котором также запущен этот файл, но не фоновом, а в привилегированном режиме. Доработать программу так, чтобы имелась возможность взаимодействия трёх и более процессов.
2. Реализовать команду `man` с помощью командного файла. Изучите содержимое каталога `/usr/share/man/man1`. В нем находятся архивы текстовых файлов, содержащих справку по большинству установленных в системе программ и команд. Каждый архив можно открыть командой `less` сразу же просмотрев содержимое справки. Командный файл должен получать в виде аргумента командной строки название команды и в виде результата выдавать справку об этой команде или сообщение об отсутствии справки, если соответствующего файла нет в каталоге `man1`.
3. Используя встроенную переменную `$RANDOM`, напишите командный файл, генерирующий случайную последовательность букв латинского алфавита. Учтите, что `$RANDOM` выдаёт псевдослучайные числа в диапазоне от 0 до

32767.

3 Выполнение лабораторной работы

Написала командный файл, реализующий упрощённый механизм семафоров.
(рис. 3.1).



```
14-1.sh - GNU Emacs at aymustafina 14-1.sh - GNU Emacs at aymustaf
File Edit Options Buffers Tools Sh-Script Help
[Icons: Open, Save, Undo, Copy, Paste, Find]
if [ "$#" -ne 3 ]; then
    echo "Использование: \$0 <t1> <t2> <номер_терминала>"
    exit 1
fi

t1=\$1
t2=\$2
terminal=\$3

semaphore_file="/tmp/semaphore.lock"

wait_for_resource() {
    echo "Процесс $$ ждет освобождения ресурса..."
    while [ -e "$semaphore_file" ]; do
        sleep 1
    done
    touch "$semaphore_file"
    echo "Ресурс занят процессом $$"
}

release_resource() {
    sleep "$t2"
    rm -f "$semaphore_file"
    echo "Процесс $$ освободил ресурс"
}

wait_for_resource

echo "Процесс $$ использует ресурс в течение $t2 секунд"
sleep "$t2"

release_resource
```

Рис. 3.1: 1

Даю права на исполнение (рис. 3.2).


```
aymustafina@aymustafina ~]$ ls -l ./14-1.sh
-rw-r--r--. 1 aymustafina aymustafina 424 мая 17 15:38 ./14-1.sh
aymustafina@aymustafina ~]$ chmod +x ./14-1.sh
aymustafina@aymustafina ~]$ ls -l ./14-1.sh
-rwxr-xr-x. 1 aymustafina aymustafina 424 мая 17 15:38 ./14-1.sh
```

Рис. 3.2: 2

Исполнение (рис. 3.3), (рис. 3.4), (рис. 3.5).

```
[aymustafina@aymustafina ~]$ ./14-1.sh 5 10 > /dev/tty2 &
[1] 92028
[aymustafina@aymustafina ~]$
```

Рис. 3.3: 3

```
[aymustafina@aymustafina ~]$ ./14-1.sh 5 10 &
[1] 92124
[aymustafina@aymustafina ~]$ Использование: $0 <t1> <t2> <номер_терминала>
```

Рис. 3.4: 4



```
14-2.sh - GNU Emacs at aymustafina
File Edit Options Buffers Tools Sh-Script Help
[Icons: Open, Save, Undo, Redo, Copy, Paste, Find]
#!/bin/bash

a=$1
if test -f "/usr/share/man/man1/$a.1.gz"
then less /usr/share/man/man1/$a.1.gz
else
echo "There is no such command"
fi
```

Рис. 3.5: 5

Реализовала команду man с помощью командного файла (рис. 3.6).

```
[aymustafina@aymustafina ~]$ chmod +x 14-2.sh
[1]+  Выход 1          ./14-1.sh 5 10 > /dev/tty2
[aymustafina@aymustafina ~]$
```

Рис. 3.6: 6

Исполнение (рис. 3.7).

```
ESC[4mESC[24m(1)
ESC[4mESC[24m(1)
User Commands
ESC[1mNAMEESC[0m
ls - list directory contents
ESC[1mSYNOPSISESC[0m
ESC[1mESC[22m[ESC[4mOPTIONESC[24m]... [ESC[4mFILEESC[24m]...
ESC[1mDESCRIPTIONESC[0m
List information about the FILES (the current directory by default). Sort entries alphabetically if none of ESC[1m-cftuvSUXESC[2
or ESC[1m-sortESC[22mis
specified.
Mandatory arguments to long options are mandatory for short options too.
ESC[1m-EESC[22m, ESC[1m--allESC[0m
do not ignore entries starting with .
ESC[1m-lESC[22m, ESC[1m--almost-allESC[0m
do not list implied . and ..
ESC[1m--authorESC[0m
with ESC[1m-lESC[22m, print the author of each file
ESC[1m-bESC[22m, ESC[1m--escapeESC[0m
print C-style escapes for nongraphic characters
ESC[1m--block-size=ESC[22m=ESC[4mSIZEESC[0m
with ESC[1m-lESC[22m, scale sizes by SIZE when printing them; e.g., '--block-size=M'; see SIZE format below
ESC[1m-EESC[22m, ESC[1m--ignore-backupsESC[0m
do not list implied entries ending with ~
ESC[1m-cESC[22mwith ESC[1m-lESC[22m: sort by, and show, ctime (time of last change of file status information); with ESC[1m-lESC[
: show ctime and sort by name; other-
```

Рис. 3.7: 7

Используя встроенную переменную \$RANDOM, написала командный файл, генерирующий случайную последовательность букв латинского алфавита (рис. 3.8).

```
14-3.sh - GNU Emacs at aymustafina
File Edit Options Buffers Tools Sh-Script Help
[Icons] Save Undo [Icons]
#!/bin/bash
a=$1
for ((i=0; i<$a; i++))
do
((char=$RANDOM%26+1))
case $char in
1) echo -n a;; 2) echo -n b;; 3) echo -n c;; 4) echo -n d;; 5) echo -n e;; 6) echo -n f;;
7) echo -n g;; 8) echo -n h;; 9) echo -n i;; 10) echo -n j;; 11) echo -n k;; 12) echo -n l;;
13) echo -n m;; 14) echo -n n;; 15) echo -n o;; 16) echo -n p;; 17) echo -n r;; 18) echo -n s;;
19) echo -n t;; 20) echo -n q;; 21) echo -n u;; 22) echo -n v;;
23) echo -n w;; 24) echo -n x;; 25) echo -n y;; 26) echo -n z;;
esac
done
echo
```

Рис. 3.8: 8

Исполнение (рис. 3.9).

```
[aymustafina@aymustafina ~]$ chmod +x 14-3.sh
[aymustafina@aymustafina ~]$ ./14-3.sh 5
upwrs
[aymustafina@aymustafina ~]$ ./14-3.sh 10
gkdqwojpqf
[aymustafina@aymustafina ~]$ █
```

Рис. 3.9: 9

4 Контрольные вопросы

1. Найдите синтаксическую ошибку в следующей строке: `while [$1 != "exit"]`

Синтаксическая ошибка: Ошибка в строке 1: `while [$1 != "exit"]` должна быть исправлена на `while [ "$1" != "exit" ]`.

2. Как объединить (конкатенация) несколько строк в одну?

Конкатенация строк: Можно использовать `string1="Hello"` и `string2="World"` и затем `combined="$string1 $string2"`.

3. Найдите информацию об утилите `seq`. Какими иными способами можно реализовать её функционал при программировании на `bash`?

Утилита `seq`: `seq` генерирует последовательность чисел. Альтернативы: циклы `for`, `while`, использование `printf`.

4. Какой результат даст вычисление выражения `$((10/3))`?

Результат выражения: Выражение `$((10/3))` даст результат 3 (целочисленное деление).

5. Укажите кратко основные отличия командной оболочки `zsh` от `bash`.

Отличия `zsh` от `bash`: `zsh` имеет более продвинутое автозаполнение, поддержку плагинов, улучшенные возможности настройки и более мощный механизм управления задачами.

6. Проверьте, верен ли синтаксис данной конструкции `1 for ((a=1; a <= LIMIT; a++))`

Проверка синтаксиса: Синтаксис конструкции `for ((a=1; a <= LIMIT; a++))` верен, если `LIMIT` определена.

7. Сравните язык `bash` с какими-либо языками программирования. Какие преимущества у `bash` по сравнению с ними? Какие недостатки?

Сравнение `bash` с другими языками: `Bash` — это скриптовый язык, удобный для автоматизации задач. Преимущества: простота и доступность для системного администрирования. Недостатки: ограниченные возможности по сравнению с полноценными языками программирования (например, отсутствие строгой типизации, сложность в отладке и ограниченная поддержка структур данных).

5 Выводы

Изучила основы программирования в оболочке ОС UNIX. Научилась писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

6 Список литературы

[lab14]